

RDMA Point-to-Point Communication for LLM Systems

Nandor Licker^{*1} Kevin Hu¹ Vladimir Zaytsev¹ Lequn Chen^{*1}

Abstract

新兴的大语言模型 (LLM) 系统模式, 如解耦推理、专家混合 (MoE) 路由以及异步强化微调, 需要超越简单集合操作的灵活点对点通信。现有实现受限于特定的网络接口控制器 (NIC), 阻碍了其集成到推理引擎中, 并限制了在不同硬件供应商间的可移植性。我们提出 *TransferEngine*, 该系统将常见 NIC 的功能进行抽象, 提供统一接口。*TransferEngine* 提供单边 WRITEIMM 操作, 并采用 IMM COUNTER 原语实现完成通知, 无需依赖网络传输的顺序性假设, 透明地管理每个 GPU 上的多个 NIC。我们在 NVIDIA ConnectX-7 和 AWS 弹性光纤适配器 (EFA) 上均实现了 400 Gbps 的峰值吞吐量。我们通过三个生产系统展示了 *TransferEngine* 的应用: (1) 用于动态缩放的解耦推理中的 KvCache 传输; (2) 实现万亿参数模型的强化学习权重更新仅需 1.3 秒; (3) MoE 调度/合并实现, 在 ConnectX-7 上的解码延迟优于 DeepEP, 且在 EFA 上首次实现了可行的低延迟。我们证明, 我们的可移植点对点通信能够与集合操作互补, 同时避免厂商锁定。

1 引言

专家混合 (MoE) 架构正成为扩展模型容量的主要方法 (Shazeer et al., 2017), 而解耦推理则正在成为生产环境服务的标准 (Patel et al., 2024; Zhong et al., 2024; Qin et al., 2025)。这些新架构依赖于与传统基于集体的并行模式根本不同的通信模式。

大型语言模型框架普遍依赖集体通信, 通常通过 NCCL 或 torch.distributed 实现。 (NVIDIA, 2015; Li et al., 2020) 尽管集体通信在静态模式 (如张量并行或数据并行) 中表现出色 (Shoeybi et al., 2020; Sergeev & Balso, 2018; Zhao et al., 2023), 但其施加的约束对新兴工作负载并不适用: 固定的成员资格限制了动态缩放, 同步初始化增加了开销, 统一的缓冲区大小迫使即使在稀疏模式下也进行稠密通信。虽然这些库提供了 SEND 与 RECV 原语以支持点对点通信, 但它们往往难以有效组合以实现可接受的延迟。

高性能计算长期使用基于远程直接内存访问 (RDMA) 的原语 (SEND、RECV、WRITE) 实现灵活的低延迟高带宽传输 (Kalia et al., 2016), 然而这类原语在大语言模型 (LLM) 框架中却极为罕见。其关键障碍在于硬件多样性缺乏统一抽象。云服务商部署了不同的 RDMA 实现: NVIDIA ConnectX 网卡采用传统的可靠连接 (RC) 传输并保证有序交付, 而 AWS 弹性网络适配器 (EFA) 则实现了专有的可扩展可靠数据报 (SRD) 协议 (Shalev et al., 2020), 支持乱序交付。现有方案普遍存在厂商锁定问题: DeepEP (Zhao et al., 2025) 要求仅由 ConnectX 提供的 GPU 发起 RDMA (IBGDA) (Agostini et al., 2018), NVSHMEM (Langer et al., 2021) 在 EFA 上表现出严重的性能下降, 而近期的库如 Mooncake (Qin et al., 2025) 和 NIXL (NVIDIA, 2025) 或缺乏 EFA 支持, 或仍处于初步阶段。因此, 目前尚无适用于多云提供商的可行 LLM 推理解决方案。

我们通过利用异构 RDMA 硬件之间的共性功能来解决可移植性问题。我们的核心见解是, ConnectX 和 EFA 均支持可靠但无序的数据传输: ConnectX

^{*}Equal contribution ¹Perplexity AI. Correspondence to: Nandor Licker <nandor@perplexity.ai>, Lequn Chen <lequn@perplexity.ai>.

RC 可以忽略顺序，而 EFA SRD 本身即为无序的。我们引入了 *TransferEngine*，一个可移植的 RDMA 通信库。它提供双向的 SEND/RECV 和单边的 WRITEIMM 操作，并引入一种新颖的 IMM-COUNTER 原语用于完成通知，该原语不依赖消息顺序。它向 ConnectX 和 EFA 暴露统一接口，避免厂商锁定。它透明地管理每个 GPU 对应的多张网卡，这对于 EFA 至关重要——必须聚合四张 100 Gbps 网卡才能达到 400 Gbps。

我们通过生产系统展示了 *TransferEngine*，以解决集体方法的不足之处：

KvCache 传输 (Section 4): 支持无限制的预填充器-解码器通信的分离推理，实现无需同步初始化或固定成员关系的弹性缩放。已在 EFA 上进行生产环境测试，支持完整的 CUDA Graph 以及低延迟的逐层传输。

强化学习权重更新 (Section 5): 点对点方法在万亿参数模型上实现了 1.3 秒的更新速度，比现有强化学习框架快超过 100 $\times$ 。(Huang et al., 2025; He et al., 2025) 通过单边 RDMA WRITE，直接从每个训练 GPU 向推理 GPU 传输，充分利用整个簇的带宽。流水线执行将 H2D memcpy、权重准备和 RDMA 传输重叠进行。

MoE 调度/合并 (Section 6): 在 ConnectX-7 上实现最先进的解码延迟，尽管使用了主机代理线程，但仍与专用的 *DeepEP* 内核性能相当。首次在 EFA 上实现可行方案，依赖并行的 token 和路由传输来隐藏设备到主机及网络延迟。

我们提出的系统涵盖了从分页写入到批量传输以及协调的散射操作等多种通信模式，均已在异构硬件上投入生产部署。我们的结果表明，点对点通信能够补充集合操作，适用于现代大语言模型工作负载，且可移植的抽象机制能够在保持性能的同时避免厂商锁定。

	RC	UC	UD	SRD	Ours
Reliability	✓	✗	✗	✓	✓
Ordering	✓	✗	✗	✗	✗
Connection	✓	✓	✗	✗	✗
SEND/RECV	✓	✓	MTU	✓	✓
WRITEIMM	✓	✓	✗	✓	✓
READ	✓	✗	✗	✓	✗
Atomic	✓	✗	✗	✓	✗

Table 1. RDMA 传输类型比较

2 背景与相关工作

2.1 网络技术

RDMA (远程直接内存访问) 远程直接内存访问 (RDMA) 是现代机器学习系统中高通量、低延迟的骨干技术。当前部署的网卡在亚 μ s 延迟下可提供 400 Gbps 的带宽。RDMA 通过分层设计实现其性能：控制平面操作由操作系统参与，用于初始化设备并设置缓冲区；而数据平面操作（数据传输和完成轮询）则绕过内核，避免了系统调用开销。

RDMA 操作 RDMA 提供了双向和单向操作。双向 SEND/RECV 操作首先使用注册的内存区域发起一个 RECV，发送方随后发出 SEND，通过完成队列通知接收方。单向 WRITE 操作将本地内存复制到远程内存区域，无需对方参与，需要远程内存地址和密钥 (RKEY)。WRITEIMM 在 WRITE 的基础上扩展，传递一个 32 比特的立即值，并通过完成队列通知接收方。尽管 READ 和原子操作可用，但其延迟不适合我们的用途。(Kalia et al., 2016; Reda et al., 2022)

RDMA 传输协议 RDMA 规范定义了三种传输协议：可靠连接 (RC)、不可靠连接 (UC) 和不可靠数据报 (UD)。Table 1 概括了它们的功能，突出了我们的 *TransferEngine* 在它们之间的共通性。

云 RDMA 适配器 除了广泛部署的 ConnectX 系列之外，主要的云服务商也在部署自己的解决方案，例如 AWS 弹性结构适配器 (EFA)、阿里云 eRDMA 以及 Google Falcon (Alibaba Cloud, 2025; Singhvi et al., 2025)。尽管大多数方案与 RC 兼容，但 EFA 实现了专有的 SRD 协议，通过 libfabric 暴露。(Shalev et al., 2020; OFI WG, 2014) SRD 是无

连接的，提供可靠但无序的数据传输。

2.2 编程接口

集合操作 机器学习框架主要依赖于集合通信库（例如 torch.distributed、NCCL、MPI）进行跨 GPU 的数据交换。（Li et al., 2020; NVIDIA, 2015; MPI Forum, 2025）虽然集合操作在结构化数据传输方面表现优异，（Shoeybi et al., 2020; Sergeev & Balso, 2018; Zhao et al., 2023）但其点对点能力有限：(1) **固定成员**阻碍了动态缩放，因为需要提前知道所有参与者；(2) **同步初始化**要求全局协调以形成“世界”，阻塞了独立的对等连接；(3) **操作顺序**要求所有参与者就操作顺序达成一致，即使 NCCL 支持并发接收，仍需额外同步；（Hu et al., 2025b）(4) **形状统一**限制了参与者之间的传输大小，即使是在点对点场景下也是如此。

GPUDirect GPUDirect RDMA 使 RDMA 网卡能够通过 PCIe 直接访问 GPU 内存，从而消除中间的 CPU 内存拷贝。（NVIDIA, 2012）传输可由主机发起，或者在支持 GPUDirect Async (IBGDA) 的情况下，由 GPU 自身发起，以绕过 PCIe 的开销（Agostini et al., 2018）。目前，GPUDirect Async 仅在 ConnectX 网卡上受支持。此外，GPUDirect RDMA 通过 GDRCopy 库实现了低延迟的主机-设备内存拷贝（Shi et al., 2014）。

2.3 相关工作

解耦推理 Splitwise、DistServe 和 Mooncake 是最早展示解耦推理架构优势的工作，它们将大语言模型推理的预填充阶段和解码阶段分离到不同的设备上。（Patel et al., 2024; Zhong et al., 2024; Qin et al., 2025）我们的 KvCache 传输用例通过将 *TransferEngine* 与推理引擎连接，利用 RDMA 实现预填充簇和解码簇之间的通信，从而实现了这一范式。

点对点网络库 NVSHMEM 同时提供了集体操作和灵活的点对点通信。（Langer et al., 2021）它支持 GPU 启动 (IBGDA) 和主机代理通信 (IBRC)。然

而，其在 EFA 上存在严重的性能下降问题。NVIDIA 推理传输库 (NIXL) 专注于大语言模型推理中的点对点通信，基于 UCX 构建 (NVIDIA, 2025; Shamis et al., 2015)。我们生产环境中部署的 EFA 实现早于 NIXL 的初步 EFA 支持 (v0.6.1, 2025 年 10 月)。Mooncake 也提供了一个 RDMA 传输引擎，但不支持 EFA。其他库，如 *UCCL* (Zhou et al., 2025) 和 *MSCCL++* (Shah et al., 2025)，侧重于集体操作的网络层最优化，而非点对点通信。

分布式 KvCache 存储 Mooncake Store 与 DeepSeek 3FS (DeepSeek AI, 2025) 提供了 KV 缓存的分布式存储，尽管它们目前尚不支持 EFA。我们通过提供适用于云部署的可移植 RDMA 原语来补充这些系统。

计算-通信重叠 针对大语言模型内核的计算与集体通信重叠的研究 (Chang et al., 2024; Zhang et al., 2025; Zheng et al., 2025a;b) 与我们关注的 RDMA 原语正交，尽管我们确实启用了后台传输。

大语言模型框架 我们的 *TransferEngine* 和 MoE 内核可集成至大语言模型推理框架及内核库，如 vLLM、SGLang、TensorRT-LLM、FlashInfer。（Kwon et al., 2023; Zheng et al., 2024; NVIDIA, 2023; Ye et al., 2025）我们的点对点权重更新方法可被强化学习框架采用，如 Slime、OpenRLHF、AResL、veRL、LlamaRL、NVIDIA Nemo。（Wu et al., 2025; Zhu et al., 2025; Mei et al., 2025; Fu et al., 2025; Sheng et al., 2024; Hu et al., 2025a）

3 TransferEngine

TransferEngine 是实现高效基于 RDMA 的点对点通信的基础库，通过简单的协议抽象了异构硬件。它暴露 SEND/RECV 操作以实现类似 RPC 的接口。对于 KvCache 传输，它提供分页 WRITE 用于批量写入。对于强化学习权重传输，它暴露低延迟高通量的 WRITE 操作。对于 MoE 路由，它针对多个对等方专门优化 WRITE 操作，以实现低延迟的 SCATTER 与 BARRIER 操作。所有操作之间均无顺序保证。可选择性地为 WRITE 关联一个立即值，接

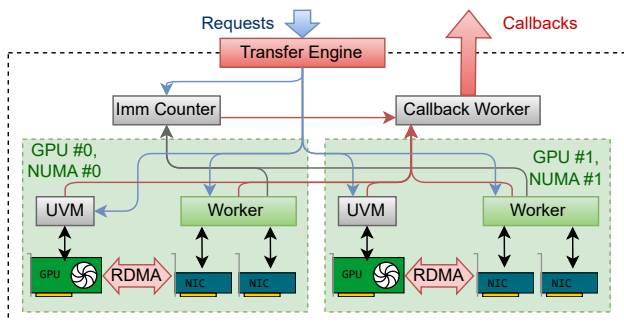


Figure 1. *TransferEngine* 管理跨 NUMA 结点的 GPU，每个结点配备多个网卡。命令被转发至工作节点，工作节点将响应返回给回调处理器或 IMM COUNTER。

收方在收到后将递增一个计数器。

3.1 概览与设计目标

TransferEngine 提供了一个最小化的 API，抽象了潜在的 RDMA 接口的复杂性。它支持多种接口，包括使用 SRD 协议的 EFA 以及通过 libibverbs 可编程的多种网卡（NIC），包括 ConnectX-7。拓扑结构可透明检测，以处理每个 GPU 配备多个 NIC 的情况：虽然单个 ConnectX-7 网卡提供 400 Gbps 带宽，但在 AWS p5 实例上实现等效带宽需要聚合四个 100 Gbps EFA 网卡（或在 p5en 实例上使用两个 200 Gbps EFA 网卡）。单个实例跨越多个线程，以管理结点内的所有 GPU 和网卡。通过零拷贝接口和硬件特定的最优化，实现了低延迟操作。

为了弥合 RDMA RC 的有序性保证与 EFA SRD 的乱序交付之间的差距，*TransferEngine* 仅依赖于一种新颖的 IMM COUNTER。不依赖顺序性保证，所有完成通知均通过轮询完成队列来处理，并在批量接收即时值后发出通知。通知通过回调函数或原子标志位进行传递。

3.2 架构

Figure 1 描绘了 *TransferEngine* 的架构。该引擎为每个 GPU 启动一个工作进程，每个进程管理一个通用的 DOMAIN GROUP，用于协调所有相关的 RDMA 网卡，通常为 1-4 块网卡，具体数量取决于硬件平台。在通用的 DOMAIN GROUP 中，每个 DOMAIN 针对特定硬件进行优化，负责单个网卡，处理队列对管理、工作提交和完成轮询。

```
#[serde] struct NetAddr(Bytes);
#[serde] struct MrDesc{ ptr: u64, rkeys: Vec<(NetAddr, u64)> }
struct MrHandle(NonNull<c_void>); type Offset = u64;
struct Pages{ indices: Vec<u32>, stride: u64, offset: Offset }
struct PeerGroupHandle(u64);
struct ScatterDst{ len: u64, src: Offset, dst: (MrDesc, Offset)}
enum OnDone { Callback(fn () -> ()), Flag(Atomic<bool>)}

trait TransferEngine {
    fn main_address() -> NetAddr;
    // Memory Region Management
    fn reg_mr(ptr: u64, len: u64, device: u64) -> (MrHandle, MrDesc);
    // Two-sided Send/Recv
    fn submit_send(addr: NetAddr, msg: &[u8], cb: fn () -> ());
    fn submit_recv(len: u64, cnt: u64, cb: fn (&[u8]) -> ());
    // One-sided Write
    fn expect_imm_count(imm: u32, count: u32, cb: fn () -> ());
    fn submit_single_write(len: u64, imm: Option<u32>,
        src: (MrHandle, Offset), dst: (MrDesc, Offset), OnDone);
    fn submit_paged_writes(page_len: u64, imm: Option<u32>,
        src: (MrHandle, Pages), dst: (MrDesc, Pages), OnDone);
    // One-sided Write to a group of peers
    fn add_peer_group(addr: Vec<NetAddr>) -> PeerGroupHandle;
    fn submit_scatter(h: Option<PeerGroupHandle>, OnDone,
        imm: Option<u32>, src: MrHandle, dst: Vec<ScatterDst>);
    fn submit_barrier(h: Option<PeerGroupHandle>, OnDone,
        imm: u32, dst: Vec<MrDesc>);
    // Watcher for CPU-GPU synchronization
    fn alloc_uvm_watcher(cb: fn(u64, u64) -> ()) -> NonNull<u64>;
}
```

Figure 2. *TransferEngine* API 伪代码。省略了错误处理、领域分片和资源释放。

每个 *TransferEngine* 实例暴露一个用于识别和发现远程对等方的单一主地址。我们使用 NetAddr 结构体来捕获并序列化 DOMAIN 的网络地址，在希望通信的对等方之间交换该结构体。作为一项限制，所有对等方必须为每个 GPU 使用相同数量的网卡（NIC）。因此，任何传输操作都能完全了解源域和目标域之间的网卡情况，使得 *TransferEngine* 可以对请求进行分片或负载均衡。这对于 EFA 至关重要，因为 EFA 通过多个适配器实现了高达 400Gbps 的带宽。

3.3 API 设计

TransferEngine 所提供的 API 在 Figure 2 中概述，实现了对 RDMA 的抽象：

内存注册 内存区域必须注册到引擎中，返回一个可序列化的 MrDesc，该标识符可与对等方交换以提交 WRITE 操作，以及一个本地的 MrHandle，用作传输的数据源。相同的 API 可用于注册主机端缓冲区和 GPU 内存。在不透明类型背后，句柄携带了它们所关联的所有网卡的地址，以及与之关联的领域特定远程密钥列表，形式为 (NetAddr, RKEY) 对。

点对点传输 submit_send 及其远程对应操作 sub-

`mit_recv` 封装了 SEND/RECV 操作，以暴露基于 RPC 的通信方式，用于交换小规模数据负载。提交操作涉及一次拷贝，以便调用者能够立即重用或释放缓冲区。接收操作会将一个循环缓冲池中的缓冲区预先提交，用于接收数据。每当收到一条消息时，会临时从池中取出一个缓冲区，使回调函数能够在不进行拷贝的情况下处理该消息。回调函数执行完毕后，缓冲区会自动重新提交回池中。必须分配足够的缓冲区数量，以避免拒绝消息。这些操作仅使用域组中的第一张网卡。

`submit_single_write` 与 `submit_paged_writes` 将数据从源缓冲区传输到目标缓冲区，分别根据间接索引、步幅和偏移量确定连续或分页的区域。该引擎将这些操作转换为可能多个零拷贝的一端 RDMA WRITE 操作，并沿可用的网卡进行分片和轮转。每个传输操作可选择携带一个 32 比特的立即数，用于在完成时通知接收方。传输完成通知以异步方式传递给调用者。

`submit_scatter` 将源缓冲区的一个数据块发送到对等组中每个对等方的不同偏移位置的接收缓冲区，而 `submit_barrier` 是仅用于对等方通知的立即操作。这些是围绕 WRITE 的优化包装，允许应用程序预先注册一个 *PeerGroup*，以实现低延迟的大批量传输。

传输在两个设备之间执行：系统中多个设备的操作协调由用户负责。

UVM 监视器 为了使主机能够在 GPU 进度推进时发起传输，`alloc_uvm_watcher` 注册了一个回调函数，该函数在内存中的某个字发生变化时被调用。它分配了一个统一虚拟内存（UVM）位置，该位置可由设备（包括 CUDA 图内的内核）更新，并由一个 CPU 线程通过 GDRCopy 持续轮询。由于并非所有更改都能立即被观察到，因此回调函数会接收旧值和新值，从而能够响应 GPU 侧的进度变化。

完成通知 通知通过回调或原子标志位，分别在发送确认和接收完成时传递。`IMMCOUNTER` 是一个专用组件，用于跟踪从潜在设备的完成队列中获取事件时递增的每个即时计数器。在传输完成后，发送

方生成事件；当带有即时数据的负载被完全交付后，接收方生成事件，从而保证原子性。计数器分配在与领域工作线程相同的 NUMA 结点上。

计数器可以通过 GDRCopy 与 GPU 透明地同步，也可以通过轮询直接观察，或在引擎内的独立专用线程中传递给回调函数，这些操作通过 `expect_imm_count` 注册。所有同步均使用此类计数器实现，因为否则无法保证其他顺序性。

3.4 实施

我们的 *TransferEngine* 采用 Rust 实现，通过精心优化内存分配、线程管理和同步机制，以最小化延迟并实现高吞吐量。该引擎为每个 DOMAINGROUP 启动一个工作线程，每个线程被固定在与设备相连的 NUMA 结点的 CPU 核心上，以同时最小化调度延迟和内存访问延迟。在完成绑定后，为特定领域分配专用的数据结构，以确保内存在正确的 NUMA 结点上预留。一个工作线程负责处理最多 4 个 DOMAIN，每个管理一个独立的网卡，而另一个专用线程则负责轮询 GPU 以更新 UVM 监视器。线程间通信通过无锁队列完成。

API 将请求转发给负责相应设备的 DOMAINGROUP，其中第一个同时负责主机。在紧密循环中，域工作线程轮询新任务，优先提交新请求。根据硬件和配置情况，请求会被分片并在可用的 DOMAIN 之间进行负载均衡。复合请求的第一个 WRITE 会立即发布到 DOMAIN 中网卡的发送队列。当新请求耗尽后，工作线程开始推进挂起的任务，通过发布写操作来填满硬件流水线。随后，轮询完成队列以查询已完成的传输并立即增加计数器。事件被聚合以提供每传输的通知，并将传输交给由所有组共享的专用回调线程。

DOMAINGROUP 内部的分片具有灵活性。传输可以按索引指定特定的网卡。单个 WRITE 可以被拆分。分页传输、分散和屏障操作均会转换为多个 WRITE，这些操作可以在所有网卡上进行分片。

3.5 硬件特定的最优化

DOMAIN 在 *TransferEngine* 中针对其所控制的硬件进行了专业化和优化：

AWS EFA 我们使用 libfabric 实现 EFA 支持，在 DOMAINGROUP 中为每个 NIC 管理一个网络接口域 (Domain)。由于 EFA 与 RDMA 规范存在差异，后者不要求仅立即写入且大小为零的传输需要有效的目标描述符，因此我们对所有传输强制要求有效的描述符。对于批量传输和对等组，我们采用工作请求 (WR) 模板化技术，在提交前预先填充并保留 libfabric 描述符中的通用字段。

NVIDIA ConnectX-7 我们通过 libibverbs 实现对 ConnectX 的支持。对于每个对端，我们使用一个无连接 (UD) 队列对来交换可靠连接 (RC) 握手信息。我们为每个对端创建两个 RC 队列对：一个用于双向 SEND/RECV 操作，另一个用于单边 WRITE 和 WRITEIMM 操作。这种分离是必要的，因为 RECV 和 WRITEIMM 的完成都会按提交顺序消耗工作请求 (WR)，这使得我们能够在不相互干扰的情况下提供高级别的 RECV 语义并支持 WRITEIMM。除了工作请求模板化之外，我们还通过将最多 4 个工作请求通过 `ibv_send_wr` 的 `next` 指针链接起来，实现工作请求链式处理，从而减少发送到网卡 (NIC) 的门铃 (doorbell) 次数。此外，我们启用 `IBV_ACCESS_RELAXED_ORDERING`，允许网卡与 GPU 内存之间的 PCIe 事务乱序执行，从而降低延迟。

4 KvCache 传输

在本节中，我们概述了一种经过生产环境验证的、依赖于 *TransferEngine* 的去中心化推理实现。在去中心化模式下，一个预填充结点对输入 token 执行预填充操作，将生成的 KV 页面以及任何额外上下文（如最后一个 token 的隐状态和用于推测解码的 Logit）传输至解码器结点，解码器结点随后逐个进行 token 解码。

图 3 展示了我们的解耦设置。一旦收到请求，全局

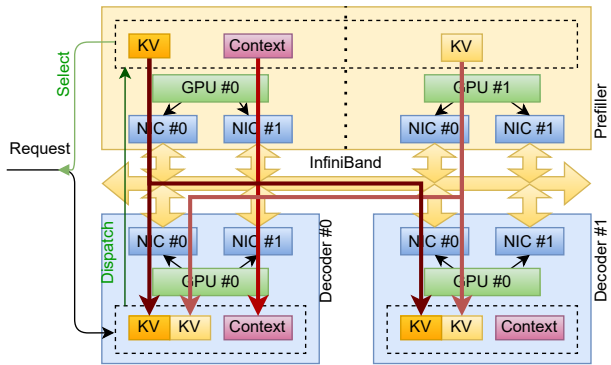


Figure 3. 预填充器与解码器之间的 KV 传输

调度器会选择一个预填充结点和一个解码器结点来处理该请求，并将请求转发给解码器。解码器会预先分配 KV 页和存储空间以容纳任何上下文，然后使用 `submit_send` 将请求分发到指定的预填充结点，同时指示应传输内容的 KV 页索引。

在分块预填充过程中，我们在每一层的注意力输出投影之后增加 UVM 监视器的值，这一操作与 CUDA Graph 兼容。一旦 *TransferEngine* 检测到 UVM 监视器值的变化，便会启动该层的传输，通过 `submit_paged_writes` 将当前分块的页复制过来。当最后一个分块完成且上下文已填充完毕后，通过 `submit_single_write` 进行复制。预填充器通过 `submit_recvs` 等待指令。

KV 传输必须考虑预填充器与解码器之间 KV 缓存分片或复制方式的差异。当使用张量并行时，MLA (Liu et al., 2024) 会复制压缩后的 KV 缓存条目：在此方案下，预填充器秩与解码器秩随机匹配，以平衡副本的传输。在 GQA (Ainslie et al., 2023) 中，我们依赖于按页偏移和步幅来从源 KV 缓存中选择片段，并复制到目标 KV 缓存的相应偏移位置。为了最小化写入次数并确保每次写入足够大，KV 缓存的布局将头置于页之前，从而保证连续头内部的连续性。

预填充器不会发出明确的完成消息：解码器事先知道它期望的传输次数，并使用 `expect_imm_count` 通知 *TransferEngine* 传输完成，然后开始解码。

生产环境中的解耦解码实现的复杂性在于对错误和取消操作的处理。解码器触发的取消必须由预填充器显式确认，因为只要存在远程写入覆盖它们的可

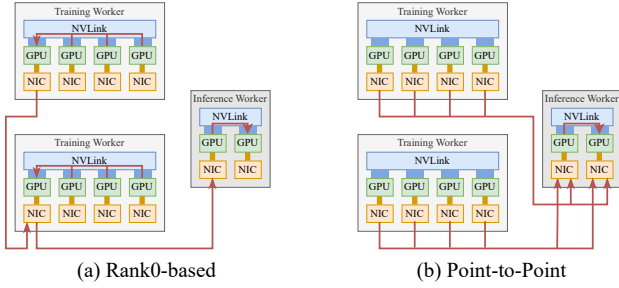


Figure 4. 不同方法的权重迁移数据路径。

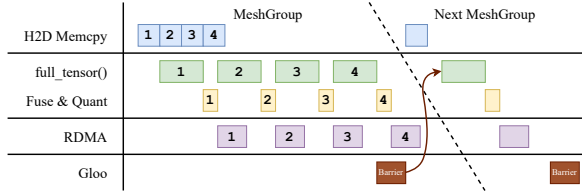


Figure 5. 流水线权重传输执行。

性能，KV 页面就无法重用。我们依赖预填充器与解码器之间的心跳消息来检测传输层故障。如果某个预填充器节点无响应，解码器在超时后将取消请求，因为数据传输已无法到达该节点。每个请求的取消 token 可以阻止所有未来的传输操作，同时等待所有正在进行的操作完成后，再发送取消确认。

5 强化学习滚动权重迁移

在异步强化学习微调中，训练和推理运行在分离的 GPU 上。每次训练步骤后，必须将新权重推送到推理结点，使用现有框架在拥有万亿级参数的模型上可能需要数十到数百秒。我们的解决方案实现了 1.3 秒内跨机器的参数更新，适用于规模达到 Kimi-K2 (1T 参数)、DeepSeek V3 (671B) 和 Qwen3 (235B) 的模型 (Kimi Team et al., 2025; DeepSeek-AI et al., 2025; Yang et al., 2025)，将权重从 256 个训练 GPU (bf16) 传输至 128 个推理 GPU (fp8)。

5.1 点对点权重传递

现有的框架倾向于为所有训练和推理 GPU 构建一个全局的集体通信世界。权重首先汇聚到训练子组的 Rank0，然后再广播到每个推理子组的 Rank0，这一过程受限于训练 Rank0 的网卡性能。相比之下，在我们的点对点 (P2P) 方法中，每个训练 GPU 通过单边 RDMA WRITE 直接将权重发送至推理 GPU，

充分利用了集群中所有网卡的带宽。Figure 4 展示了两种方法之间的差异。

在初始化时，控制器脚本执行三个步骤：首先，它从所有训练和推理 GPU 收集参数元数据，包括权重名称、形状、数据类型以及 DTensor 分片信息。接着，它计算一个静态权重传输调度方案，确定哪个训练 GPU 向哪个推理 GPU 发送哪些参数，以及传输顺序。最后，它将该调度方案广播到所有训练 GPU。

在每个训练步骤中，控制器会发出信号，指示训练 GPU 开始发送权重。推理结点对这一传输过程毫无察觉，因为该过程使用的是单边操作。

5.2 流水线权重传输执行

我们的训练任务使用 FSDP (Zhao et al., 2023) 对模型权重进行划分。不同类型的参数（例如，MoE 与非 MoE）需要采用不同的 FSDP 划分策略。每种划分策略将全局 DeviceMesh 划分为互不重叠的子网格。我们将每个子网格称为 MeshGroup。同一 MeshGroup 内的参数并行传输，而 MeshGroups 之间则按顺序处理。

我们将每个参数张量的传输视为一个任务。为了同时利用不同的硬件资源，我们将每个任务拆分为四个流水线阶段，这些阶段在时间上重叠，如 Figure 5 所示：(1) 主机到设备的 memcopy (当 FSDP 将权重卸载到 CPU 时)。(2) 参数准备：使用 full_tensor() 重构完整权重，应用投影融合，必要时进行量化。(3) RDMA 传输：零拷贝 WRITE 到远程推理 GPU 内存。(4) 全局屏障：在所有 full_tensor() 调用完成后，通过以太网使用 GLOO 在网格组间同步。

full_tensor() 和其他 GPU 操作会引入额外的 GPU 内存使用。为避免内存不足错误，我们仅在当前进行中的任务占用的临时 GPU 内存低于可配置的水印时才启动新任务。

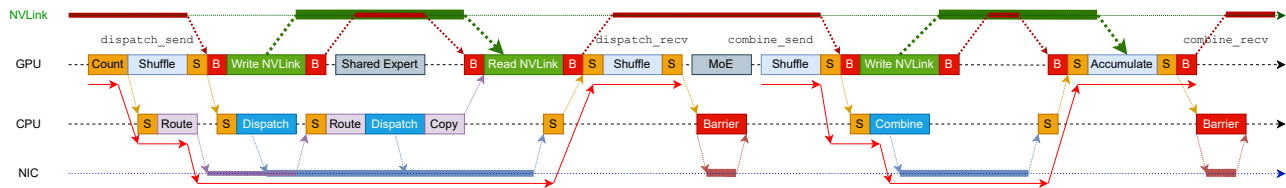


Figure 6. 调度与 GPU-CPU-NIC 协同

6 MoE 路由/合并

我们提出了一套低延迟的内核，用于 MoE 调度和合并，其核心是 *TransferEngine*，依赖于主机代理线程来协调 GPU 和网卡。在结点内部，我们还利用 NVLink 来减轻网络负载。尽管增加了代理线程的延迟，我们仍实现了业界领先的解码性能，同时在无需任何调整的情况下保持了预填充阶段具有竞争力的性能。

这些内核展示了基于代理的 MoE 调度的可行性，并支持更广泛的网络卡类型，例如 EFA。因此，我们重点关解解码性能（每秩 128 个 token），因为该性能受延迟限制，在涉及的设备间额外的 PCIe、驱动程序和固件开销下受到更显著的影响。

6.1 架构

路由通过解码和合并的拆分内核实现，发送方部分将数据准备到发送缓冲区以写入对端，接收方部分则将接收缓冲区中的 token 重新排列为其他内核使用的张量。各个内核充分利用了 GPU 的所有 SM 和内存带宽以降低延迟，但其运行时间足够短，可在它们之间交错执行其他工作，从而实现重叠和微批量处理。主机代理使用 GDRCopy 轮询 GPU 的进度，在源缓冲区就绪时调用 *TransferEngine*。

我们的设计如图 6 所示，通过减少写入次数来最小化代理开销。为降低发送和接收缓冲区所需的 GPU 内存，所有对等结点首先交换路由信息和每专家的 token 数量，以确定一个唯一值域，用于写入连续的接收缓冲区。由于这些数据量较小，其延迟可通过将少量 token 预调度到每个源的私有缓冲区来隐藏。合并操作仅需一次散射，因为它复用了路由信息。路由信息始终由 *TransferEngine* 处理，但数据负载可以在同一结点内通过 NVLink 复制。因此，每个秩

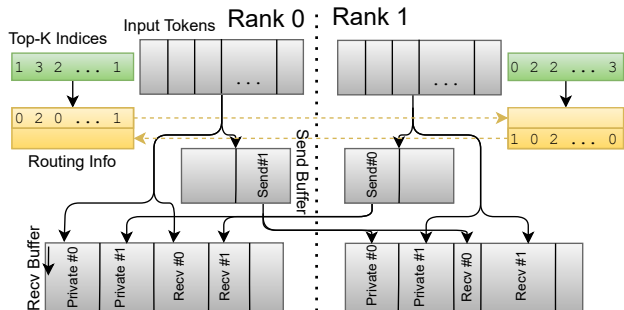


Figure 7. 分派到私有且连续的缓冲区

在每次跨结点对等通信中最多发出 2 次 WRITE 用于分发，以及 1 次 WRITE 用于合并。

接收缓冲区的大小必须足以容纳发送到当前秩的所有 token。假设有 N 个秩托管 E 个专家，每个专家向 R 个专家分发 T 个 token，则上限为 $N \cdot T \cdot \max(R, \frac{E}{N})$ 。发送方将写入打包到这种连续缓冲区中，而不是依赖于每个秩较大的接收缓冲区。为了最小化开销，较大的分发接收缓冲区可以被合并发送重复使用。

6.2 派遣

调度内核接收 token 以及它们必须被路由到的专家的 R 索引，返回一个张量，该张量打包了分配给本地专家的所有秩的 token。内核首先在共享内存中统计发送到每个专家的 token 数量，并通过统一内存将计数传输到主机。然后，它向代理发出信号，代理使用 *TransferEngine* 启动将路由信息散播到所有对等秩。随后，输入 token 被复制到发送缓冲区，创建一个连续的数据源以散播到每个对等秩，如图 7 所示。

代理被通知将 token 散布到每个接收方对等节点的私有缓冲区中，直到达到固定限制。对于解码大小的批量数据，从分发内核启动到首次传输的延迟约为 $15 \mu s$ （假设 $EP=64$ ）。此时，路由和 token 的传输已

饱和网卡带宽。一旦接收到路由，并在每个目标秩上确定每个源秩的 token 位置后，其余 token 将被散列到对等节点，并连续放置到共享接收缓冲区中。主机端处理路由并调度第二轮传输的工作耗时数十微秒。初始传输的 token 数量被选择以隐藏此开销。尽管该延迟不在关键路径上，但减少它可进一步减小私有缓冲区的大小。当所有传输均被确认完成且所有传入写入均已记录后，通过 *TransferEngine* 的屏障同步代理。

在通过 RDMA 向跨结点对等方发送传输后，发送内核继续通过 NVLink 发起写操作，以在同一个结点对内传输 token，同时后台仍在进行 RDMA 传输。NVLink 对等方通过其自身的屏障机制进行同步：在向对等方写入之前，每个秩必须确保数据已被读取且可以被覆盖。这通过使用 relaxed 语义写入和读取标志位来实现。在数据负载写入完成后，各秩之间通过 release-acquire 标志位进行同步。

发送内核的执行顺序是延迟敏感的，因为内存屏障的粒度不足。NVLink 通过虚拟内存暴露，透明地将映射到当前地址空间的对端设备的读写操作转换为互连上的事务。加载操作普遍代价高昂，因为它们会阻塞执行流水线，直到操作完成。相比之下，存储操作是“发出即忘”的，直到遇到内存屏障时才会阻塞，此时需等待其作用域内所有先前的存储操作完成。由于主机系统和 NVLink 对端均处于同一作用域内，确保与主机顺序一致的屏障可能会被之前通过 NVLink 发出的写操作拖慢。为避免这种情况，采用先向主机发送信号，再在网格屏障之后发出 NVLink 写操作的策略。该策略虽然增加了发送内核的总执行时间，但显著降低了关键路径上首次 RDMA 传输的延迟。

通过 NVLink，通常更优的做法是从源设备向目标设备推送数据，从而节省一次往返时间。此外，在当前设备上确认存储操作后，可以在远程传输进行的同时执行有用的工作。在分派过程中，我们仅将 token 推送到私有接收缓冲区，因为在这一阶段，尚无法获取集中式路由信息，无法确切确定其余 token 应放置在对端的何处。内核的接收端通过同步屏障

并读取剩余 token 来启动。这些加载操作很可能在 RDMA 操作完成之前就已结束。

发送分发后，传输操作在后台运行。一旦调用接收内核，它将等待 *TransferEngine* 通过 IMMCounter 和 GDRCopy 报告所有传输完成。基于路由信息计算出的索引，token 按照适合分组 GEMM 内核的布局进行重新排序，并可在专家之间选择性地添加填充。由于接收内核必须在关键路径上处理约 $T \cdot R$ 个 token，因此工作会被分配到所有可用的 SM 上，并最大化流水线以充分利用可用的 HBM 带宽。

6.3 结合

由于在调度阶段路由信息是集成的，因此合并操作将所有负载通过一次散射完成。在发送与合并之间的空闲时间内准备命令，此时 GPU 通常会执行分组 GEMM。发送方类似于调度过程，准备发送缓冲区，并通过 NVLink 将 token 推送至同节点的对等方。主机代理被触发，向 TRANSFERENGINE 发送 Scatter 请求，之后内核执行完毕。接收方在等待所有 token 到达之前，将从路由信息中推导出的所有相关偏移量缓存至共享内存中。随后，每个秩在本地计算 token 的加权平均值。

合并阶段重用了分发阶段的相同缓冲区，因此需要同时使用 NVLink 和 RDMA 屏障，以确保在覆盖发送缓冲区之前完成所有先前的操作。此时，主机代理也等待 *TransferEngine* 确认所有写入操作均已发出。特别是对于 EFA，它还需要等待接收确认，因此最大化提交写入与检查其状态之间的时间间隔非常重要。

6.4 与 DeepEP 的比较

DeepEP 提供了最先进的延迟性能，然而由于其依赖于 IBGDA 和 mlx5 驱动程序，因此与 ConnectX 紧密绑定。我们的内核具有更高的可移植性，依赖于主机代理，同时支持 EFA 和 ConnectX。尽管存在额外的开销，我们的延迟性能仍优于 DeepEP。

DeepEP 内核依赖于 RC QP 强大的顺序保证。token 在可用的 SM 之间均衡分布，并通过 QP 逐个传输。

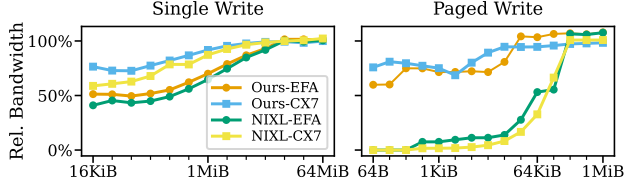


Figure 8. 点对点通信性能

这确保了首次传输的延迟更低，但同时也增加了每个 token 的工作量，并导致在网络上传输更多的数据包。token 数量和完成情况通过 ATOMIC 信号通知。相比之下，由于 PCIe 上额外的 GPU-CPU-NIC 通信开销，我们的内核在首次传输启动前需要花费更多时间。然而，批量传输实现了更高的网络利用率。

对于预填充阶段，我们采用单次传输策略并进行缩放，无需额外调整。相比之下，DeepEP 通过在发送结点上利用 NVLink 预先累积 token，降低了传输的数据量，从而实现了更低的延迟。此外，DeepEP 内核在传输 token 子集时采用批量方式，因此使用的缓冲内存更少。尽管该方法速度更快，但也对准确率和确定性产生影响，因为累积过程并非完全以固定顺序在 fp32 上完成。

7 评估

我们评估了 *TransferEngine* 以及基于它的系统在配备 8× NVIDIA H200 GPU 的结点上的性能，这些 GPU 通过 NVLink 连接，并且每个 GPU 配备单个 400 Gbps ConnectX-7 或双 200 Gbps EFA 网络接口卡，所有设备均连接到双路 Intel Sapphire Rapids CPU。

		EFA	CX-7
Single	64 KiB	16 Gbps	44 Gbps
	256 KiB	54 Gbps	116 Gbps
	1 MiB	145 Gbps	245 Gbps
	32 MiB	336 Gbps	378 Gbps
Paged	1 KiB	17 Gbps 2.11M op/s	91 Gbps 11.10M op/s
	8 KiB	138 Gbps 2.10M op/s	320 Gbps 4.89M op/s
	16 KiB	274 Gbps 2.08M op/s	367 Gbps 2.80M op/s
	64 KiB	364 Gbps 0.69M op/s	370 Gbps 0.71M op/s

Table 2. EFA 与 ConnectX-7 性能对比

7.1 点对点通信

KvCache 传输和 RL 回放的性能由单次写入和分页写入的吞吐量决定。我们对比了更高层级的库 (*TransferEngine* 与 *NIXL* v0.6.1) 与底层网卡的硬件基准，测量了在一系列消息大小下的带宽占比。在 ConnectX 上，我们以 `ib_write_bw` (来自 *rdma-core*) 作为对比基准；在 EFA 上，我们使用 `fi_rma_bw` (来自 *libfabric*) 来测量单网卡峰值带宽，并将其翻倍以匹配运行在两个网卡上的引擎性能。

Figure 8 显示了两个库所达到的峰值带宽比例。使用单个 WRITE 时，需要至少 16MiB 的消息才能达到饱和，而使用分页 WRITE 时，32KiB 和 64KiB 的消息分别足以在 *TransferEngine* 和 *NIXL* 上实现饱和。我们的方案与 *NIXL* 之间的性能相对接近，其中 *TransferEngine* 稍快一些。EFA 需要更大的消息才能达到饱和，这解释了在 MoE 路由中观察到的差距。

Table 2 详细说明绝对性能数值。对于 256 KiB 的单个 WRITE (我们 MoE 路由的典型情况)，我们的 *TransferEngine* 在 EFA 上实现了 54 Gbps，而在 ConnectX-7 上达到了 116 Gbps。在 64 KiB 分页 WRITE (KV 缓存页的典型大小) 情况下，EFA 和 ConnectX-7 均能够达到可用带宽的饱和状态。RL 权重传输消息的大小远超饱和点。

7.2 MoE 路由/合并

我们评估了 MoE 内核在 8、16、32 和 64 块 GPU 上的解码和预填充性能，将其与目前性能最优的开源实现 DeepEP 进行比较，同时也与基于 NVSHMEM v3.4.5 构建的可移植且开源的 *pplx-kernels* (Licker et al., 2025) 进行对比，后者运行在 EFA 和 ConnectX-7 上。尽管我们的工作与 UCCL-EP (Mao et al., 2025) 存在部分重叠，但我们未进行详细比较，因为其公布的延迟值显著更高。我们表明，在 ConnectX-7 网卡上，即使使用主机代理，我们的性能已达到新的状态水平。

为了与 DeepEP 进行对比，我们的基准测试考虑了 DeepSeek-V3 模型的设置。我们调度 $7168 \times \text{fp8}$ 个

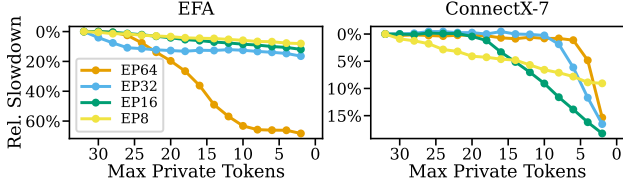


Figure 9. 私有缓冲区大小对 p50 解码延迟的影响

token, 采用 $56 \times \text{fp32}$ 个缩放因子, 并合并相同维度的 bf16 张量。解码在批量为 128 的情况下进行评估, 预填充在每块 4096 个 token 的数据上进行。每个 token 被分派给 8 个随机专家。我们执行 10,000 次预热迭代, 随后进行 10,000 次基准测试运行, 并对所有秩的耗时进行聚合。我们通过插入大型 GEMM 来模拟重叠计算并清除缓存。

7.2.1 私有缓冲区大小

私有缓冲区的设计旨在隐藏路由信息交换的延迟。我们比较了在不同 token 数量下, 总解码分发时间的中位数延迟, 以及在缓冲区大小足以一次性容纳所有 token 时所达到的峰值延迟。

Figure 9 表示随着私有缓冲区大小减小, 性能下降的情况。在没有 token 的情况下性能退化, 证明了采用延迟降低策略的合理性。在节点内 (intra-node) 场景中, 由于 token 传输速度较快, 至少需要约 32 个 token 才能隐藏跨两个网卡的路由交换延迟。在节点间 (inter-node) 场景中, ConnectX-7 网卡允许仅使用 24 个 token, 而 EFA 网卡在 token 数量低于 32 时便已出现性能下降, 这是因为路由交换速度较慢。

7.2.2 发送和接收延迟

我们的内核同样像 DeepEP 一样分为发送方和接收方, 两者之间的 NVLink 和 RDMA 传输在后台执行。Figure 10 展示了这些延迟, 这些延迟是通过插入一个长的人工延迟 (模拟共享专家或重叠工作) 在接收内核之前测量得到的, 以确保所有传输都已稳定。

分发和合并发送优于 DeepEP, 因为我们的方法仅复制内存。由于累积速度更快, 合并接收也更快。分发接收是异常值, 因为它使用 NVLink 加载数据。

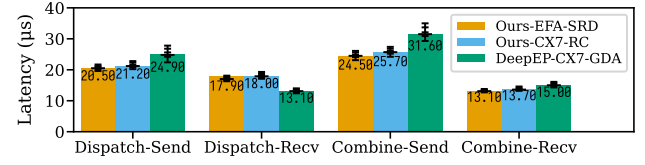


Figure 10. EP=64 的发送与接收延迟分离

总体而言, 这些内核的总执行时间不到传输时间的 15%。更深入的分析显示, 代理在发送内核执行中途即开始进行 RDMA 操作, 而数据重排仅增加了约 15 μs 的空闲时间。

7.2.3 解码延迟

Figure 11 展示了我们在解码规模批量下的 MoE 调度和合并内核的延迟, 将其与专用的 DeepEP 以及可移植的 *pplx-kernels* 进行比较。我们仅报告了 ConnectX-7 上的 *pplx-kernels* 结果, 因为 EFA 上的 NVSHMEM 性能极差, 无法使用。我们的基于代理的实现不仅使 EFA 变得可行, 而且在 ConnectX-7 上显著快于基于 IBGDA 的内核, 并且比通过通用主机代理使用 IBRC 的 NVSHMEM 内核快一个数量级。

在使用 8 个秩的结点内设置中, 我们的内核比 DeepEP 慢约 2 μs , 这主要是由于使用网卡交换路由信息所致。尽管该数据点对我们的主要跨结点应用场景并非关键, 但它凸显出: 通常被 RDMA 隐藏的 NVLink 传输延迟非常低, 效率极高。 *pplx-kernels* 在此配置下依赖于细粒度的 per-token NVLink 同步。这种差异源于通过 NVLink 进行的大批量传输以及 per-block/per-grid 同步。

在 16 和 32 个秩的情况下, 我们的内核在分发和合并阶段均优于 DeepEP, 这主要归因于批量传输以及合并阶段的高效流水线处理。RDMA 与 NVLink 写入的顺序也有助于降低首次 RDMA 传输的延迟。当扩展到 64 个秩时, 合并阶段仍然优于 DeepEP, 但代理线程的 CPU 开销变得明显, 且由于需为 56 个节点间对等端分别排队一个约微秒级的传输操作, 导致分发阶段变慢。

由于解码未饱和带宽, 尽管 256KiB 写入的吞吐量不足 ConnectX-7 的百分之五十, EFA 延迟仅落后

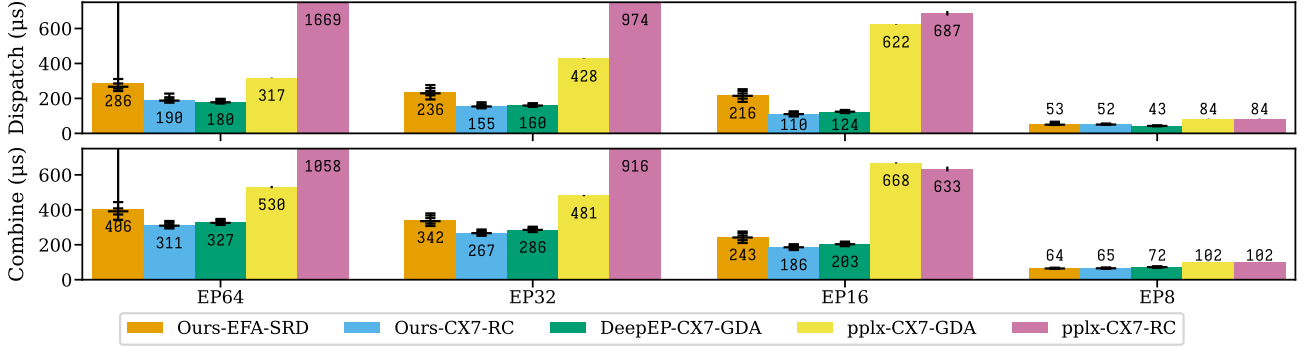


Figure 11. MoE 解码延迟。条形高度表示均值。误差条显示 p01、p25、p50、p75、p95、p99。pplx 的误差条表示标准差。

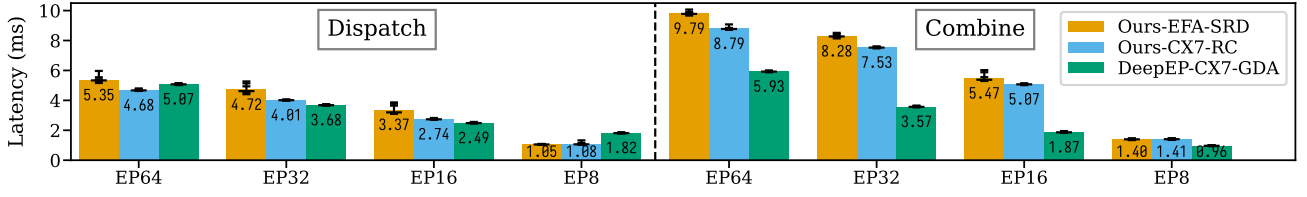


Figure 12. MoE 预填充延迟。柱状图高度表示均值。误差条显示 p01、p25、p50、p75、p95、p99。

30%。

7.2.4 预填充延迟

Figure 12 显示了 MoE 预填充的延迟。我们排除了 *pplx-kernels* 的比较，因为它们 4096 个 token 时效果不佳。由于传输过程中缺乏分块机制，我们解码优化内核的内存开销限制了可行部署的模型集合。

在调度方面，DeepEP 在较少的秩上表现更优，因为它们使用 RDMA 仅传输一个 token 副本，通过 NVLink 将该副本的拷贝传送到其他秩。在 EP64 上，多个副本落在同一节点的可能性较小，因此这种最优化的效果逐渐减弱。在 combine 场景下，DeepEP 的发送端部分求和显著减少了 RDMA 字节数，因此尽管累积精度降低至 bf16，仍表现出更低的延迟。

8 结论

现有的用于大语言模型系统的 RDMA 解决方案存在厂商锁定问题，缺乏在自定义云硬件（如 AWS EFA）上的可行实现。*TransferEngine* 通过识别异构 RDMA 硬件间的共性功能来解决此问题。通过对潜在协议构建一个无顺序保证的可靠抽象层，我们透明地扩展了对多种 RDMA 网卡的支持，特别

聚焦于 EFA 和 ConnectX。

我们通过三个生产系统展示了这一方法：用于解耦推理的 KvCache 传输、实现万亿参数模型 1.3 秒级强化学习权重更新，以及在 ConnectX-7 上达到业界领先延迟的 MoE 调度/合并，并实现了首个与 AWS EFA 兼容的可行方案。*TransferEngine* 为现代大语言模型架构提供了可移植的点对点通信能力，在避免厂商锁定的同时，与集合通信库协同，支持云原生部署。

致谢

我们感谢 NVIDIA 慷慨提供 ConnectX-7 硬件以供我们的评估，并提供了关于如何在 H200 GPU 上最大化性能的宝贵见解。我们感谢 AWS 在提升 EFA 性能方面提供的洞察与建议。

参考文献

Agostini, E., Rossetti, D., and Potluri, S. GPUDirect async: Exploring GPU synchronous communication techniques for InfiniBand clusters. *J. Parallel Distributed Comput.*, 114:28–45, 2018. doi: 10.1016/J.JPDC.2017.12.007. URL <https://doi.org/10.1016/J.JPDC.2017.12.007>.

[//doi.org/10.1016/j.jpdc.2017.12.007](https://doi.org/10.1016/j.jpdc.2017.12.007).

Ainslie, J., Lee-Thorp, J., De Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

Alibaba Cloud. Elastic compute service: eRDMA, 2025. URL <https://www.alibabacloud.com/help/en/ecs/user-guide/elastic-rdma-erdma/>.

Chang, L.-W., Bao, W., Hou, Q., Jiang, C., Zheng, N., Zhong, Y., Zhang, X., Song, Z., Jiang, Z., Lin, H., Jin, X., and Liu, X. Flux: Fast software-based communication overlap on gpus through kernel fusion, 2024.

DeepSeek AI. Fire-flyer file system (3fs). <https://github.com/deepseek-ai/3FS>, 2025. High-performance distributed file system for AI training and inference workloads.

DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T.,

Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., and Pan, Z. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.

Fu, W., Gao, J., Shen, X., Zhu, C., Mei, Z., He, C., Xu, S., Wei, G., Mei, J., Wang, J., Yang, T., Yuan, B., and Wu, Y. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL <https://arxiv.org/abs/2505.24298>.

He, B., Zhu, Z., and Li, J. Efficient rl training - optimizing weight sync in slime. <https://hebiao064.github.io/rl-weight-sync>, 2025. Accessed: 2025-10-14.

Hu, J., Wu, X., Shen, W., Liu, J. K., Zhu, Z., Wang, W., Jiang, S., Wang, H., Chen, H., Chen, B., Fang, W., Xianyu, Cao, Y., Xu, H., and Liu, Y. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework, 2025a. URL <https://arxiv.org/abs/2405.11143>.

- Hu, Z., Shen, S., Bonato, T., Jeauegy, S., Alexander, C., Spada, E., Dinan, J., Hammond, J., and Hoefer, T. Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms, 2025b. URL <https://arxiv.org/abs/2507.04786>.
- Huang, G., Chadha, P., Kong, T., Gao, W., and Li, Z. NeMo-RL: Journey of optimizing weight transfer in large moe models by 10x. <https://github.com/NVIDIA-NeMo/RL/discussions/1189>, 2025. Accessed: 2025-10-14.
- Kalia, A., Kaminsky, M., and Andersen, D. G. Design guidelines for high performance RDMA systems. In Gulati, A. and Weatherspoon, H. (eds.), *Proceedings of the 2016 USENIX Annual Technical Conference, USENIX ATC 2016, Denver, CO, USA, June 22-24, 2016*, pp. 437–450. USENIX Association, 2016. URL <https://www.usenix.org/conference/atc16/technical-sessions/presentation/kalia>.
- Kimi Team, Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., Wang, H., Wang, J., Wang, J., Wang, J., Wang, S., Wang, S., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, Q., Wu, W., Wu, X., Wu, Y., Xiao, C., Xie, X., Xiong, W., Xu, B., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, Y., Xu, Z., Yan, J., Yan, Y., Yang, X., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Yao, X., Ye, W., Ye, Z., Yin, B., Yu, L., Yuan, E., Yuan, H., Yuan, M., Zhan, H., Zhang, D., Zhang, H., Zhang, W., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zheng, H., Zheng, S., Zhou, J., Zhou, X., Zhou, Z., Zhu, Z., Zhuang, W., and Zu, X. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stolica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Langer, A., Howell, S., Potluri, S., Dinan, J., and Kraus, J. Dynamic symmetric heap allocation in NVSHMEM. In *OpenSHMEM and Related Technologies. OpenSHMEM in the Era of Exascale and Smart Networks: 8th Workshop on OpenSHMEM and Related Technologies, OpenSHMEM 2021, Virtual Event, September 14–16, 2021, Revised Selected Papers*, pp. 187–198, Berlin, Heidelberg, 2021. Springer-Verlag. ISBN 978-3-031-04887-6. doi: 10.1007/978-3-031-04888-3_12. URL https://doi.org/10.1007/978-3-031-04888-3_12.

- Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., and Chintala, S. PyTorch Distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020. doi: 10.14778/3415478.3415530. URL <http://www.vldb.org/pvldb/vol13/p3005-li.pdf>.
- Licker, N., Hu, K., Zaytsev, V., and Chen, L. pplx-kernels: Perplexity MoE kernels. <https://github.com/perplexityai/pplx-kernels>, 2025.
- Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Mao, Z., Zhou, Y., Zhang, Y., Cui, C., Chen, Z., and Xu, Z. Previewing UCCL-EP: Flexible and efficient expert parallelism for cloud and beyond. <https://uccl-project.github.io/posts/uccl-ep/>, 2025.
- Mei, Z., Fu, W., Li, K., Wang, G., Zhang, H., and Wu, Y. Real: Efficient rlhf training of large language models with parameter reallocation. In *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. mlsys.org, 2025.
- MPI Forum. *MPI: A Message-Passing Interface Standard Version 5.0*, June 2025. URL <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>.
- NVIDIA. GPUDirect RDMA, 2012. URL <https://docs.nvidia.com/cuda/gpudirect-rdma/>.
- NVIDIA. NVIDIA collective communication library (NCCL), 2015. URL <https://developer.nvidia.com/nccl>.
- NVIDIA. TensorRT-LLM. <https://github.com/ai-dynamo/nixl>, 2023.
- NVIDIA. NIXL: NVIDIA inference xfer library. <https://github.com/ai-dynamo/nixl>, 2025.
- OFI WG. libfabric: Open Fabrics Interfaces (OFI), 2014. URL <https://github.com/ofiwg/libfabric>.
- Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*, pp. 118–132. IEEE, 2024. doi: 10.1109/ISCA59077.2024.00019. URL <https://doi.org/10.1109/ISCA59077.2024.00019>.
- Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, Santa Clara, CA, February 2025. USENIX Association. ISBN 978-1-939133-45-8. URL <https://www.usenix.org/conference/fast25/presentation/qin>.
- Reda, W., Canini, M., Kostic, D., and Peter, S. RDMA is turing complete, we just did not know it yet! In Phanishayee, A. and Sekar, V. (eds.), *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, pp. 71–85. USENIX Association, 2022. URL <https://www.usenix.org/conference/nsdi22/presentation/reda>.
- Sergeev, A. and Balso, M. D. Horovod: fast and easy distributed deep learning in tensorflow, 2018. URL <https://arxiv.org/abs/1802.05799>.

- Shah, A., Jangda, A., Li, B., Rocha, C., Hwang, C., Jose, J., Musuvathi, M., Saarikivi, O., Cheng, P., Zhou, Q., Dathathri, R., Maleki, S., and Yang, Z. Msccl++: Rethinking gpu communication abstractions for cutting-edge ai applications, 2025. URL <https://arxiv.org/abs/2504.09014>.
- Shalev, L., Ayoub, H., Bshara, N., and Sabbag, E. A cloud-optimized transport protocol for elastic and scalable HPC. *IEEE Micro*, 40(6):67–73, 2020. doi: 10.1109/MM.2020.3016891. URL <https://doi.org/10.1109/MM.2020.3016891>.
- Shamis, P., Venkata, M. G., Lopez, M. G., Baker, M. B., Hernandez, O., Itigin, Y., Dubman, M., Shainer, G., Graham, R. L., Liss, L., et al. UCX: an open source framework for HPC network APIs and beyond. In *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*, pp. 40–43. IEEE, 2015.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q. V., Hinton, G. E., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- Shi, R., Potluri, S., Hamidouche, K., Perkins, J. L., Li, M., Rossetti, D., and Panda, D. K. Designing efficient small message transfer mechanism for inter-node MPI communication on InfiniBand GPU clusters. In *21st International Conference on High Performance Computing, HiPC 2014, Goa, India, December 17-20, 2014*, pp. 1–10. IEEE Computer Society, 2014. doi: 10.1109/HIPC.2014.7116873. URL <https://doi.org/10.1109/HiPC.2014.7116873>.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training multi-billion parameter language models using model parallelism, 2020. URL <https://arxiv.org/abs/1909.08053>.
- Singhvi, A., Dukkupati, N., Chandra, P., Wassel, H. M. G., Sharma, N. K., Rebello, A., Schuh, H., Kumar, P., Montazeri, B., Bansod, N., Thomas, S., Cho, I., Seibert, H. L., Wu, B., Yang, R., Li, Y., Huang, K., Yin, Q., Agarwal, A., Vaduvatha, S., Wang, W., Moshref, M., Ji, T., Wetherall, D., and Vahdat, A. Falcon: A reliable, low latency hardware transport. In Curado, M., Rothenberg, C. E., Porter, G., and Kandula, S. (eds.), *Proceedings of the ACM SIGCOMM 2025 Conference, SIGCOMM 2025, São Francisco Convent, Coimbra, Portugal, September 8-11, 2025*, pp. 248–263. ACM, 2025. doi: 10.1145/3718958.3754353. URL <https://doi.org/10.1145/3718958.3754353>.
- Wu, B., Wang, S., Tang, Y., Ding, J., Helenowski, E., Tan, L., Xu, T., Gowda, T., Chen, Z., Zhu, C., Tang, X., Qian, Y., Zhu, B., and Hou, R. LlamaRL: A distributed asynchronous reinforcement learning framework for efficient large-scale LLM training, 2025. URL <https://arxiv.org/abs/2505.24034>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao,

- R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025. URL <https://arxiv.org/abs/2501.01005>.
- Zhang, S., Zheng, N., Lin, H., Jiang, Z., Bao, W., Jiang, C., Hou, Q., Cui, W., Zheng, S., Chang, L.-W., Chen, Q., and Liu, X. COMET: Fine-grained computation-communication overlapping for mixture-of-experts. In *Eighth Conference on Machine Learning and Systems*, 2025. URL <https://openreview.net/forum?id=fGgQS5VW09>.
- Zhao, C., Zhou, S., Zhang, L., Deng, C., Xu, Z., Liu, Y., Yu, K., Li, J., and Zhao, L. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025.
- Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16(12):3848–3860, August 2023. ISSN 2150-8097. doi: 10.14778/3611540.3611569. URL <https://doi.org/10.14778/3611540.3611569>.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.
- Zheng, S., Bao, W., Hou, Q., Zheng, X., Fang, J., Huang, C., Li, T., Duanmu, H., Chen, R., Xu, R., Guo, Y., Zheng, N., Jiang, Z., Di, X., Wang, D., Ye, J., Lin, H., Chang, L.-W., Lu, L., Liang, Y., Zhai, J., and Liu, X. Triton-distributed: Programming overlapping kernels on distributed ai systems with the triton compiler, 2025a. URL <https://arxiv.org/abs/2504.19442>.
- Zheng, S., Fang, J., Zheng, X., Hou, Q., Bao, W., Zheng, N., Jiang, Z., Wang, D., Ye, J., Lin, H., et al. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives. *arXiv preprint arXiv:2503.20313*, 2025b.
- Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In Gavrilovska, A. and Terry, D. B. (eds.), *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pp. 193–210. USENIX Association, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
- Zhou, Y., Chen, Z., Mao, Z., Lao, C., Yang, S., Kannan, P. G., Gao, J., Zhao, Y., Wu, Y., You, K., Ren, F., Xu, Z., Raiciu, C., and Stoica, I. An extensible software transport layer for gpu net-

working, 2025. URL <https://arxiv.org/abs/2504.17307>.

Zhu, Z., Xie, C., Lv, X., and slime Contributors.
slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025.
GitHub repository. Corresponding author: Xin Lv.